

UNIVERSITY OF ENGINEERING AND TECHNOLOGY, LAHORE
DEPARTMENT OF ELECTRICAL ENGINEERING

Programming for AI | Fall 2026

Flight and Airport Analytics (SkyLens)

Submitted by
BS.AI Fall 2025, Section C

Muhammad Usman	2025-AI-167
Abdul Rehman	2025-AI-155
M Saim Akram	2025-AI-146
Taha Abdul Rehman	2025-AI-274

Instructor: Mr. Shaban G. Satti
Submission date: November 2026

Abstract

This report documents the final ETL and EDA implementation of **Flight and Airport Analytics (SkyLens)**, version 2.1.1. The pipeline **extracts** live flight data from the AviationStack REST API, **validates** every record with Pydantic, **transforms** it with pandas, **loads** it into a SQLite database in one atomic transaction, **verifies** the stored data with SQL checks, and then performs a structured **exploratory data analysis** that reads exclusively from the database. A PySide6 desktop application with 14 pages and a terminal menu present the same data.

The delivered database holds **230 flights** and **134 airports**, built from four archived API responses (235 records). Pydantic accepted 232 records and rejected 3 that carry no flight identifier; the 232 became 230 unique flights after two overlaps between responses were merged, and re-running the pipeline inserts no new rows. Cross-checking the API's `live.updated` field showed that its flight times are airport-local times mislabelled as UTC; the pipeline converts them with each airport's own timezone, which removes impossible flight durations.

The EDA shows that only 54.3% of flights report a delay value; of those, 22.4% were delayed, with a median delay of 0 min, a mean of 4.0 min and a maximum of 68 min (strongly right-skewed). Operated flights were delayed more often than codeshared listings (40% versus 7%, $p < 0.001$), an association that is discussed with its caveats. The code base is covered by 276 automated tests that all pass. The report states its limitations plainly: the data is a three-date snapshot, delay data is partial, and the live APIs could not be called during final verification, so extraction was tested with mocked responses and archived real responses.

Table of Contents

1 Introduction **4**
1.1 Background and Motivation 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope and Report Organisation 4
2 Requirements **5**
2.1 Functional Requirements 5
2.2 Non-Functional Requirements 5
3 Data Source and Dataset **5**
3.1 AviationStack Flights API 5
3.2 Plan Limits and Raw Archive 6
3.3 Dataset Delivered 6
4 Tools and Technologies **7**
5 System Architecture and Design **7**
5.1 Layered Architecture 7
5.2 Module Overview and Design Principles 8
6 ETL Pipeline Implementation **9**
6.1 Workflow and Run History 9
6.2 Extraction and Error Handling 9
6.3 Validation with Pydantic 10
6.4 Transformation with pandas 12
6.5 Database Design and Loading 13
7 Exploratory Data Analysis **14**
7.1 Methodology 14
7.2 Dataset Overview and Data Quality 14
7.3 Descriptive Statistics 15
7.4 Distributions 15
7.5 Trends 16
7.6 Relationships and Comparisons 16
7.7 Outliers 17
7.8 Key Findings 17
8 Application Interfaces **18**
8.1 Desktop Application 18
8.2 Responsive Layout and Performance 20
9 Testing and Verification **21**
9.1 Automated Tests 21
9.2 End-to-End Verification 21
9.3 What Could Not Be Verified 21
10 Limitations **21**
11 Future Enhancements **22**
12 Conclusion **22**
References **22**
Appendix A: Reproduction Instructions **23**

1 Introduction

1.1 Background and Motivation

Air travel produces a continuous stream of operational data: scheduled and actual times, delays, terminals, gates, aircraft identifiers and live positions. Commercial providers expose this data through web APIs, but a raw JSON response is nested, partly empty and not directly suitable for analysis. Turning it into trustworthy information requires a systematic Extract–Transform–Load (ETL) process followed by exploratory data analysis (EDA).

SkyLens is a Python application that implements this complete cycle on real flight data. Its pipeline extracts flight records from the AviationStack REST API [1], validates every record with Pydantic [2], cleans and enriches it with pandas [3], loads it into a SQLite database [4] in a single atomic transaction, verifies the stored data with SQL checks, and then analyses it. The same stored data feeds a PySide6 [5] desktop application with 14 pages (dashboards, flight search, route map, weather, monitoring and reports) and a terminal menu. This report describes version 2.1.1 of the project.

1.2 Problem Statement

A raw API response cannot simply be trusted. Fields may be missing, values may be malformed or impossible, the same flight can be returned more than once, timestamps may be mislabelled, and a free plan limits how often data can be downloaded. The project therefore needs a pipeline that (i) fetches data and survives network and API failures, (ii) rejects invalid records while explaining why, (iii) stores clean data without duplicates and without ever leaving a half-written batch, (iv) can be re-run safely, and (v) supports an honest exploratory analysis of what was actually stored.

1.3 Objectives

- **Extract** flight records from the AviationStack `/v1/flights` endpoint with filters, handling HTTP errors, timeouts, empty or malformed responses and API-level failures, with the access key kept in environment variables only.
- **Validate** every incoming record and every transformed row with Pydantic models and report accepted records, rejected records and field-level errors.
- **Transform** the data with pandas (missing values, duplicates, data types, inconsistent formats, invalid records), justifying every decision.
- **Load** the data into SQLite with a defined schema, keys, constraints, parameterised SQL and transactional integrity, so that re-running never duplicates data.
- **Analyse** the stored data with a structured EDA (dimensions, types, missing values, duplicates, statistics, distributions, trends, relationships, comparisons, outliers) and communicate it with labelled charts and cautious findings.
- **Engineer** the solution as clear modules with logging, configuration, automated tests and reproducible commands, and expose it through a desktop application.

1.4 Scope and Report Organisation

SkyLens is a single-user tool working on a modest snapshot of live data obtained within the quotas of a free API plan. It does not predict delays, does not claim causes and does not attempt a historical time series (Section 10). Section 2 lists the requirements, Section 3 describes the data, and Sections 4 and 5 describe the technologies and architecture. Section 6 documents the ETL implementation, Section 7 the exploratory analysis and Section 8 the application. Testing, limitations, future enhancements and the conclusion follow in Sections 9 to 12.

2 Requirements

2.1 Functional Requirements

Table 1. Functional requirements and the modules that realise them.

ID	Requirement	Realised in
FR1	Extract flight records with filters and paging; survive timeouts, HTTP errors and malformed responses	`api_client.py`
FR2	Keep credentials out of the code and out of logs, GUI and error messages	`config.py`, `.env`, `_redact()`
FR3	Validate raw and transformed records; report accepted and rejected records with reasons	`models.py`
FR4	Clean, convert and derive columns with documented, justified decisions	`data_processor.py`, `timeutils.py`
FR5	Load into SQLite with keys and constraints in one atomic transaction; re-runs must not duplicate	`database.py`
FR6	Verify the stored data with SQL integrity checks	`database.verify_database()`
FR7	Structured EDA from the database with labelled charts and generated findings	`eda.py`, `eda_charts.py`, notebook
FR8	Interactive application: dashboards, delay analytics, airport and airline views, route map, weather, flight search	`src/gui/`, `providers/`
FR9	Monitoring, reports and export (CSV, Excel, PDF) of exactly the selected rows	`etl_monitor.py`, `reports.py`
FR10	Reproducible without an API key or quota	`data/raw/`, `--replay-raw`, SAMPLE mode

2.2 Non-Functional Requirements

- **Reliability and integrity:** transient failures are retried, failed runs are logged, loads are all-or-nothing and the database enforces CHECK, UNIQUE and FOREIGN KEY constraints.
- **Security:** the access key is read from the environment, shown masked, and removed from every error message.
- **Transparency:** every record shown in the application is labelled by origin (LIVE, HISTORICAL or SAMPLE DATA); sample data is never presented as real.
- **Responsiveness and portability:** the GUI stays responsive through caching and deferred chart drawing and adapts from desktop to phone-width windows; the code targets Python 3.10+ and ships a Dockerfile.
- **Testability:** the whole code base is covered by automated tests that need neither network access nor an API key.

3 Data Source and Dataset

3.1 AviationStack Flights API

The data source is the AviationStack flights endpoint (`/v1/flights`) [1]. It was chosen because it delivers real operational data as JSON through a simple query-string interface with an access key, offers a free plan, and returns a nested payload with many optional fields, which makes validation and cleaning meaningful rather than trivial. Each request is an HTTP GET with the parameters `access_key`, `limit`, `offset` and optional filters (`flight_status`, `dep_iata`, `arr_iata`, `airline_name`, `flight_number`). The response contains a pagination object and a `data` array whose elements are structured as shown in Table 2.

Table 2. Structure of one API record and how the pipeline uses it.

Block	Representative fields	Use in the pipeline
flight_date, flight_status	ISO date; scheduled, active, landed, cancelled, incident, diverted	natural key; status analysis
departure / arrival	airport name, IATA, ICAO, terminal, gate, baggage, timezone, delay, scheduled / estimated / actual times	routes, delays, durations, local hour
airline	name, IATA (2 characters), ICAO	airline ranking and comparison
flight	number, IATA, ICAO, codeshared (object or null)	identity; codeshare flag
aircraft (optional)	registration, ICAO24	sparse descriptive fields
live (optional)	latitude, longitude, altitude, speed, ground flag, updated	tracking fields (16 flights); `live.updated` is the evidence for the timestamp finding

Three further sources support the application but are **not** part of the ETL pipeline or the EDA: FlightAPI [6] supplies flight options for Flight Search (a clearly labelled SAMPLE provider is used without a key), Open-Meteo [7] supplies weather, and two bundled public-domain files, OurAirports [8] and Natural Earth [9], supply airport coordinates (all 134 stored airports resolve to coordinates) and the land outline of the offline route map.

3.2 Plan Limits and Raw Archive

The free plan allows roughly 100 requests per month and about 100 records per request. Every successful response is therefore written untouched to `data/raw/` (four files holding 100, 25, 10 and 100 records, retrieved on 9 and 13 October 2026). The API reported 10,000 matching records for the unfiltered query; only the retrieved records form the dataset. The archive makes the pipeline reproducible without spending quota: `run_pipeline.py --replay-raw` re-runs validation, transformation and loading on these real responses.

3.3 Dataset Delivered

The delivered database holds **230 flights** on 3 flight dates (2026-10-09, 2026-10-10 and 2026-10-13), 134 derived airports, 101 distinct airline names (99 airline codes), 127 routes and 26 departure timezones. By status, 184 flights are scheduled, 45 active and 1 landed. The `flights` table has 46 columns: an identifier, 43 data columns and two audit timestamps. Of the 235 raw records, 121 are codeshare listings of other carriers' flights.

4 Tools and Technologies

Table 3. Technologies and their role in the project.

Technology	Role
Python 3.10+ [10]	implementation language (the Docker image uses 3.12); `zoneinfo` and `sqlite3` from the standard library [15]
requests [11]	HTTP client with timeouts, retries and error mapping (`api_client.py`)
python-dotenv [12]	loads `AVIATIONSTACK_API_KEY` and settings from `.env`; the key never appears in code
Pydantic v2 [2]	two validation gates: `FlightRecord` for raw records, `FlightRow` for transformed rows
pandas, NumPy [3]	flattening, cleaning, type conversion, derived columns and EDA computations
SQLite [4]	persistent store with upsert, CHECK / UNIQUE / FOREIGN KEY constraints and transactions
Matplotlib [13]	all charts (EDA figures and GUI charts)
PySide6 [5]	desktop GUI with 14 pages and an animated aviation theme
openpyxl	Excel export through pandas
pytest [14]	276 automated tests
Jupyter / nbconvert	`notebooks/ETL_EDA_Analysis.ipynb`, stored with its outputs
Docker	`Dockerfile` and `docker-compose.yml` for tests, pipeline and (via X11) the GUI

Note. `scikit-learn` is listed in `requirements.txt` but is not imported by any module of the project; no machine-learning model is part of the delivered code.

5 System Architecture and Design

5.1 Layered Architecture

The code is split into three layers (Figure 1). The **presentation** layer (command-line runners, terminal menu and GUI) contains no data logic and only calls the application layer. The **application** layer orchestrates the pipeline and the analyses. The **data** layer holds the API client, the Pydantic models, the pandas transformations, the providers and the SQLite access. Because `service.refresh_data()` is a thin wrapper around `pipeline.run_pipeline()`, the command line, the terminal menu and the GUI all execute exactly the same ETL steps.

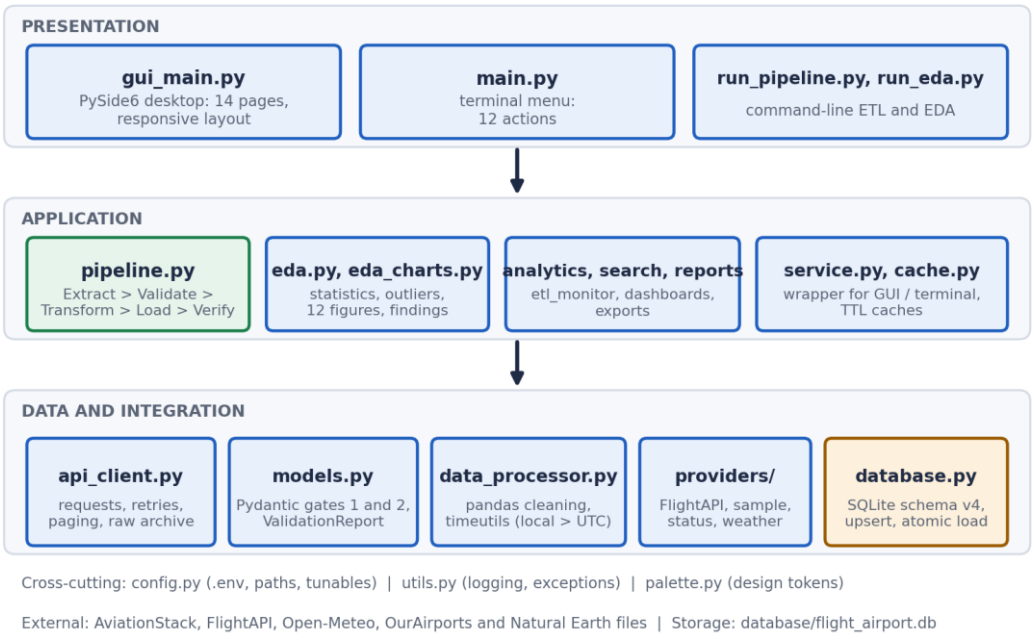


Figure 1. Layered architecture of SkyLens v2.1.1.

5.2 Module Overview and Design Principles

Table 4. Source modules with line counts from the delivered code.

Module	Lines	Responsibility
`config.py`	224	paths, `.env` loading, tunables (timeout, retries, limits)
`api_client.py`	345	requests, retries with back-off, API error mapping, pagination, raw archive, replay loader, key redaction
`models.py`	406	Pydantic gates 1 and 2, `ValidationReport`, `RejectedRecord`
`timeutils.py`	72	airport-local to UTC conversion, timezone resolution
`data_processor.py`	454	flatten, clean, derive columns, `TRANSFORM_DECISIONS`, `TransformReport`
`pipeline.py`	308	five-stage orchestration, `PipelineReport`, raw replay
`database.py`	910	schema v4, migration, upsert, atomic `load_batch`, `verify_database`
`eda.py`, `eda_charts.py`	572, 316	structured EDA, outliers, z-test, generated findings, 12 figures
`providers/` (7 files)	658	FlightAPI client, sample provider, live status, Open-Meteo weather
`flight_search`, `reports`, `etl_monitor`, `cache`, `reference`	650	search pipeline, CSV / Excel / PDF export, run monitor, TTL caches, airport reference
`src/gui/` (15 files)	5,572	pages, theme, animated scenes, widgets, render queue

In total `src/` contains 12,023 lines (including the GUI), `main_task/` 964 lines and `tests/` 2,982 lines. Four design principles run through the code. A **single orchestrator** (`run_pipeline`) serves every entry point. Every stage returns a **report object** (`ValidationReport`, `TransformReport`, `PipelineReport`) that is also saved as JSON. Cleaning decisions are kept as **data** (`TRANSFORM_DECISIONS`), so the documentation cannot drift from the code. **Derived tables** (airports, airlines, routes) are rebuilt from the stored flights inside the load transaction, so their tallies always agree with the flights table.

6 ETL Pipeline Implementation

6.1 Workflow and Run History

One call to `run_pipeline()` performs five stages and returns a `PipelineReport` with the counts of every stage; the report is saved as JSON in `outputs/reports/` and summarised on the terminal. Figure 2 annotates the stages with the counts of the delivered database.

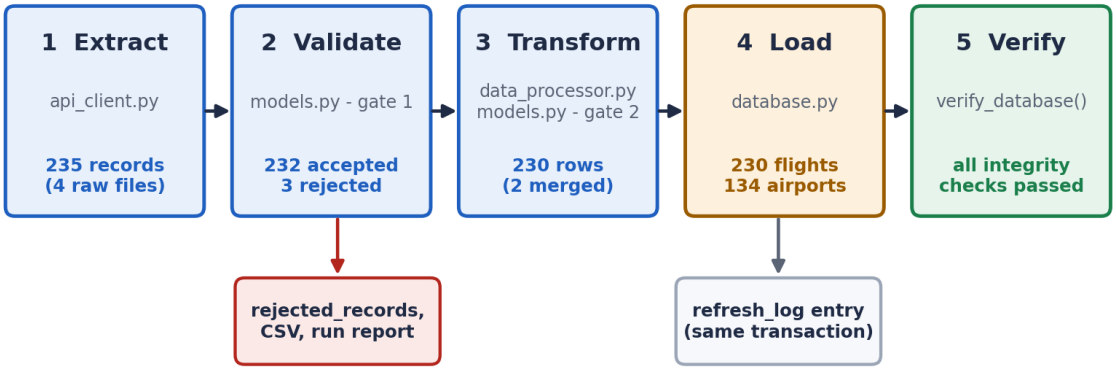


Figure 2. ETL workflow annotated with the counts obtained when the four archived responses are replayed.

Table 5 lists the nine runs recorded in the `refresh_log` table. Runs 1 to 5 were live retrievals during development; runs 6 to 9 rebuilt the database from the archive with the final pipeline. Run 4 failed because the API refused the request (HTTP 403); this is why the provider's error bodies are now parsed and shown to the user.

Table 5. Pipeline runs recorded in `refresh_log` ("-" = run pre-dates the inserted / updated counters).

Run	Source	Fetched	Valid	Rejected	Inserted	Updated	Status
1	live API, 9 Oct	100	100	0	-	-	success
2	live API, 9 Oct	25	25	0	-	-	success
3	live API, 9 Oct	10	10	0	-	-	success
4	live API, 11 Oct	0	0	0	-	-	failed (HTTP 403)
5	live API, 13 Oct	100	100	0	-	-	success
6	archive 20261009_101516	100	100	0	100	0	success
7	archive 20261009_102110	25	25	0	24	1	success
8	archive 20261009_102257	10	10	0	9	1	success
9	archive 20261013_064308	100	97	3	97	0	partial

6.2 Extraction and Error Handling

The extraction stage (`api_client.py`) never lets a bad response travel further down the pipeline. Transient problems are retried with exponential back-off (2 s, 4 s, 8 s by default, at most `MAX_RETRIES` = 3), whereas credential and quota errors fail immediately because retrying them would only waste

quota. Every failed run is written to `refresh_log` with status `failed` and the error message, so failures are visible in the audit trail.

Table 6. Failure situations and the behaviour of the extraction stage.

Situation	Behaviour	Retry?
Timeout (default 20 s), connection error, HTTP 500 / 502 / 503 / 504	`APIError` with a clear message once the retries are used up	yes, back-off
HTTP 401 / 403, invalid or missing key, usage limit (429), plan restriction	the provider's own error code and message are parsed from the body and raised as a specific exception	no
Empty body; body that is not JSON (for example an HTML error page)	"API returned an empty response body" / "API response was not valid JSON"	no
JSON without a `data` list, or `data` of the wrong type	explicit structure error; `data: null` is treated as an empty list	no
Valid response with zero records	run status `empty`, logged; nothing is loaded	-
Missing API key	configuration error before any request is made	-

Listing 1. Retry loop of `api_client._request` (abridged and re-wrapped).

```
for attempt in range(config.MAX_RETRIES + 1):
    try:
        response = requests.get(
            url, params=query,
            timeout=(min(5, config.REQUEST_TIMEOUT), config.REQUEST_TIMEOUT))
    except requests.exceptions.Timeout:
        last_problem = f"request timed out after {config.REQUEST_TIMEOUT}s"
    except requests.exceptions.ConnectionError:
        last_problem = "could not reach aviationstack.com - check your connection"
    except requests.exceptions.RequestException as exc:
        raise APIError(f"HTTP request failed: {_redact(exc)}")
    else:
        if response.status code not in  RETRYABLE STATUS:   # 500/502/503/504
            break
        last_problem = f"server error HTTP {response.status_code}"
    if attempt < config.MAX_RETRIES:
         sleep before retry(attempt)          # base * 2**attempt seconds
    else:
        raise APIError(
            f"API request failed after {config.MAX_RETRIES + 1} attempts: "
            f"{last_problem}.")
```

Credential management. The key is read from `AVIATIONSTACK_API_KEY` in the environment or `.env`; `.env` is excluded by `.gitignore` and `.env.example` documents all variables with an empty key. The key is shown masked in the GUI and terminal. Exception messages of the `requests` library can contain the complete URL including `?access_key=...`, which would leak the key into logs and the GUI; the function `_redact()` therefore removes it from every error text, and a test (`test_api_key_never_appears_in_error_messages`) proves it. **Pagination** is handled by `fetch_flights_paged()`, which uses `offset`, stops early on a short page and is capped by `max_pages`, because each page costs one request of a small monthly quota.

6.3 Validation with Pydantic

Validation happens twice. **Gate 1** (`FlightRecord` with the nested models `AirportPointInfo`, `AirlineInfo`, `FlightInfo` and `LiveInfo`) validates the raw API record before anything is transformed. **Gate 2** (`FlightRow`) validates each flattened row after pandas has processed it, so that a transformation bug can never write a malformed row to the database. Unknown extra fields are ignored and missing optional fields are allowed.

Table 7. Validation rules of gate 1.

Field / scope	Rule
flight_status	one of scheduled, active, landed, cancelled, incident, diverted (controlled vocabulary)
flight_date; timestamps	valid ISO date; valid ISO-8601 timestamps (bad text is rejected early)
airport IATA / ICAO; airline IATA	exactly 3 / 4 / 2 letters or digits, upper-cased, blank becomes `None`
delay (minutes)	0 ≤ delay ≤ 2880 (48 h); negative or absurd delays are source errors and are not clipped
live latitude / longitude	−90 to 90 / −180 to 180
record identity	flight IATA, ICAO or number must exist, otherwise the record cannot be keyed
airports	a departure or arrival airport code must be present
scheduled times	arrival not before departure, timezone-aware; if a timezone is missing a 26-hour slack is allowed

Gate 2 additionally checks 3-letter airport codes, delays in range, `scheduled_duration_min` between 0 and 24 h, `dep_local_hour` between 0 and 23, coordinates, speed between 0 and 1,500 km/h, and that `is_delayed` is true only when a positive delay is recorded.

Listing 2. Identity rules inside `FlightRecord._check_consistency` (abridged).

```
if not (self.flight.iata or self.flight.icao or self.flight.number):
    raise ValueError(
        "flight has no IATA code, ICAO code or number - cannot be identified")
if not (self.departure.iata or self.arrival.iata):
    raise ValueError("neither departure nor arrival airport code is present")
```

`validate_batch()` returns a `ValidationReport` with the accepted records, the `RejectedRecord` objects (index, identifier, stage, field-level errors and the raw payload), the acceptance rate and an error breakdown per field. Rejections are written to the `rejected_records` table, to `outputs/reports/rejected_records_*.csv`, to the run report and to the terminal. On the real archive, **235 records were extracted, 232 (98.7%) passed gate 1 and 3 were rejected** because they carry no flight IATA code, ICAO code or number (all three in the response of 13 October); gate 2 rejected none.

Because the real archive contains only this one kind of invalid record, `demo_validation.py` corrupts **copies** of real records in 11 different ways (synthetic defects used only for this test and never stored in the project database): unknown status, non-ISO date, negative delay, delay above 48 h, wrong-length airport code, wrong-length airline code, non-ISO timestamp, latitude out of range, no flight identifier, no airport code, and arrival before departure. All **11 of 11 defects were detected** (16 records submitted, 5 accepted, 11 rejected). The test also exposed a real gap in an earlier ordering rule, an arrival 26 years before departure being accepted when the arrival airport had no timezone; the rule now falls back to the 26-hour comparison and a regression test covers it.

6.4 Transformation with pandas

The transformation stage (`data_processor.py`) flattens the nested records into a 46-column table. Every cleaning decision is stored in `TRANSFORM_DECISIONS`; Table 8 summarises the main ones.

Table 8. Main transformation decisions and their justification.

Issue	Decision	Justification
Blank text; airport and airline codes	trim; blank becomes NULL; upper-case codes	one representation of "missing"; "jfk" and "JFK" are one airport
Timestamps	airport-local wall-clock time converted to true UTC with the airport's timezone; unparseable or timezone-less becomes NULL	the API labels local time with a spurious +00:00 (Section 6.4.1)
Missing airport timezone	filled only from another record of the same airport; malformed values such as "+8" read as fixed offsets	nothing is guessed; unresolved cases are counted
Delay minutes	numeric; missing values are not imputed	missing means "not reported", not "on time"
Duplicate flights (same natural key)	keep the last occurrence	later data has the most recent status and delay
`is_delayed`	true only for a reported delay above 0	unknown delay is false; the EDA reports delay coverage separately
`scheduled_duration_min`, `dep_local_hour`	derived: scheduled arrival minus departure in UTC; hour of the scheduled departure on the airport clock	block time for distributions and outliers; time-of-day needs no timezone lookup

6.4.1 Discovery: the API's "UTC" timestamps are local times

While examining implausible flight durations (for example 435 minutes from Doha to Colombo), the raw responses were tested against the one genuine UTC value they contain, `live.updated`. For all **11 flights** that carry both `live.updated` and an actual departure time, trusting the +00:00 suffix places the live report between 7.2 and 8.7 hours **before** the aircraft left, which is impossible, whereas reading the same digits as the departure airport's local time places it **18 to 48 minutes after** departure (Figure 3). The pipeline therefore discards the offset, attaches each airport's IANA timezone [16] and converts to true UTC. After the correction every duration is plausible (minimum 20 min, maximum 735 min; Doha–Colombo becomes 285 min). The raw JSON files are never modified.

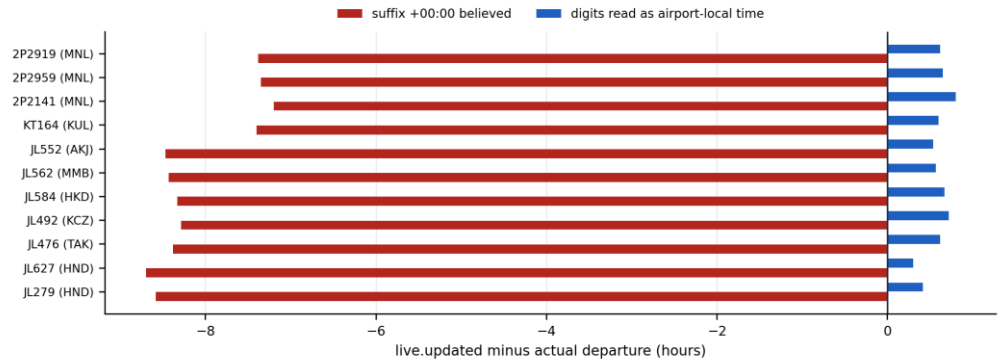


Figure 3. Evidence for the timestamp finding: 11 flights, reproduced by `eda.timestamp_semantics_check` (data in `outputs/eda/timestamp_evidence.csv`).

Listing 3. Timestamp correction in `timeutils.py` (docstring and type hints omitted).

```
def wall_clock_to_utc(value, tz_name):
    naive = parse_wall_clock(value)          # drops the spurious +00:00
    tz = resolve_timezone(tz_name)           # IANA name or fixed offset such as "+8"
    if naive is None or tz is None:
        return None                          # never guess
    return naive.replace(tzinfo=tz).astimezone(timezone.utc)
```

Honest scope of the proof: the 11 testable flights all depart from Asia (UTC+8 and UTC+9). The conversion is applied to every airport because it is the same API field everywhere and the corrected

durations are plausible, but flights from other regions were not independently proven. After filling two timezones from other records of the same airport, **43 of 230 flights** still lack a timezone at one or both airports; their timestamps stay NULL, which is why `scheduled_duration_min` exists for 187 of 230 flights. The transformation stage reports rows in and out, duplicates removed, timezones filled and missing values per column in a `TransformReport`; on the archive 232 validated records became 230 rows because two flights occur in more than one response and were merged.

6.5 Database Design and Loading

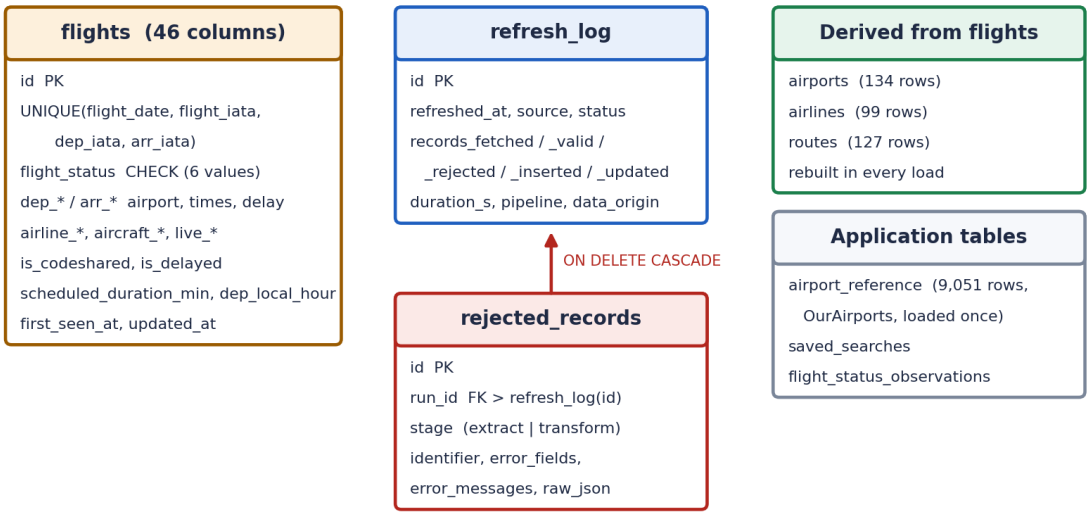


Figure 4. Database schema (SQLite, schema version 4).

Table 9. Database design choices.

Aspect	Implementation
Primary keys	`id INTEGER PRIMARY KEY AUTOINCREMENT` on every table (the reference table is keyed by IATA code)
Natural key	`UNIQUE(flight_date, flight_iata, dep_iata, arr_iata)`; missing key parts are stored as empty text because SQLite treats NULLs as distinct inside UNIQUE constraints
Constraints	`CHECK` on flight_status (6 values), delays ≥ 0, flags 0 / 1, local hour 0–23, rejection stage; `PRAGMA foreign_keys = ON`
Foreign key	`rejected_records.run_id` references `refresh_log(id)` with `ON DELETE CASCADE`
Audit columns	`first_seen_at` (kept on re-runs), `updated_at`, `retrieved_at`
Derived tables	airports, airlines and routes are rebuilt from all stored flights in every load
Safe SQL	parameterised queries; identifiers that cannot be parameterised are checked against allow-lists

`load_batch()` runs inside one `with connect()` block: upsert of the flights, rebuild of the derived tables, insertion of the `refresh_log` row and insertion of the rejected records. If any statement fails, the whole transaction is rolled back, so the database never contains a half-loaded batch (test `test_load_is_atomic`). The function returns how many rows were **inserted** and how many already existed and were **updated**; the original row id and `first_seen_at` survive re-runs.

Listing 4. True upsert in `database.py` (abridged).

```
sql = (f"INSERT INTO flights ({', '.join(all_cols)}) VALUES ({placeholders}) "
       f"ON CONFLICT({', '.join(UNIQUE_KEY)}) DO UPDATE SET {set_clause}")
conn.executemany(sql, rows)
```

To demonstrate idempotency, the four archived responses were replayed twice into a fresh database (Table 10). The second pass inserts nothing: every record is recognised and refreshed, the table stays at 230 flights and the integrity checks pass. The same demonstration is in the notebook.

Table 10. Idempotency evidence: two passes over the archive.

Raw file	Records	Accepted	Pass 1 ins / upd	Pass 2 ins / upd	Flights after pass 2
20261009_101516	100	100	100 / 0	0 / 100	230
20261009_102110	25	25	24 / 1	0 / 25	230
20261009_102257	10	10	9 / 1	0 / 10	230
20261013_064308	100	97	97 / 0	0 / 97	230

`verify_database()` then runs real SQL queries against the stored data (Table 11). Rows migrated from an older schema keep their old, uncorrected timestamps; the command `--replay-raw --rebuild` regenerates them from the archive. A database created by an earlier version is detected on first connection (`PRAGMA table_info`) and upgraded in one transaction to schema version 4 without losing flights or run history (tests cover the migrations from versions 1 and 3).

Table 11. Integrity checks on the delivered database (`PRAGMA user_version` = 4).

Check	Result
`PRAGMA integrity_check`; foreign-key violations	ok; 0
Flights; duplicate natural keys (GROUP BY … HAVING COUNT > 1)	230; 0
Rows without a flight identity; invalid status; negative delay	0; 0; 0
Airport sightings versus 2 × flights with airports	460 versus 460
All checks passed	true

7 Exploratory Data Analysis

7.1 Methodology

The EDA (`src/eda.py`, `src/eda_charts.py`) follows the classical exploratory approach [17] and begins with `database.fetch_flights()`: it analyses the **stored** data, not an in-memory API response. It is presented in `notebooks/ETL_EDA_Analysis.ipynb` (executed, outputs stored) and produces 12 figures, `eda_summary.json` and `EDA_Findings.md`; all numbers below are taken from these artefacts, and re-running `run_eda.py` reproduces them identically. The analysis proceeds through nine steps: overview (shape, types, date coverage); data quality (missing values, exact and natural-key duplicates, constant columns, delay coverage); descriptive statistics (mean, standard deviation, quartiles, skew); distributions (bar charts and histograms); trends (by flight date and local departure hour); relationships (Spearman rank correlation with the number of rows behind each cell, and a two-proportion z-test); comparisons (airlines, departure airports, status); outliers (Tukey 1.5 × IQR rule); and findings generated by code from the computed numbers, so the text cannot drift from the data.

Four choices are worth stating. Spearman correlation is used because delays are heavily skewed. A missing delay is excluded from delay rates instead of being counted as on time. For delays the IQR of the raw column is zero (most flights report 0), so the outlier fence is computed on delayed flights only. The z-test is implemented with the standard library to avoid an extra dependency.

7.2 Dataset Overview and Data Quality

The dataset has **230 rows and 47 columns** when loaded for analysis (46 stored columns plus the derived local departure hour): 26 text, 9 floating-point, 7 date-time, 3 integer and 2 boolean columns. There are **no exact duplicate rows and no natural-key duplicates**; the only constant columns are the two audit timestamps. Flights are spread unevenly over the three flight dates: 132, 1 and 97 on 9, 10 and 13

October. Arrival gate, aircraft registration, arrival baggage and the live-tracking fields are 80–99% empty and the departure terminal and gate about half empty (Figure 5, left), presumably because the API fills them only when that information exists. More important is that **125 of 230 flights (54.3%)** have a delay value, so all delay results describe this subset. Arrival delay exists for only 18 flights, so departure delay carries the analysis. Only three statuses occur (Figure 5, right): 80.0% scheduled, 19.6% active and one landed flight, so status has little discriminating power.

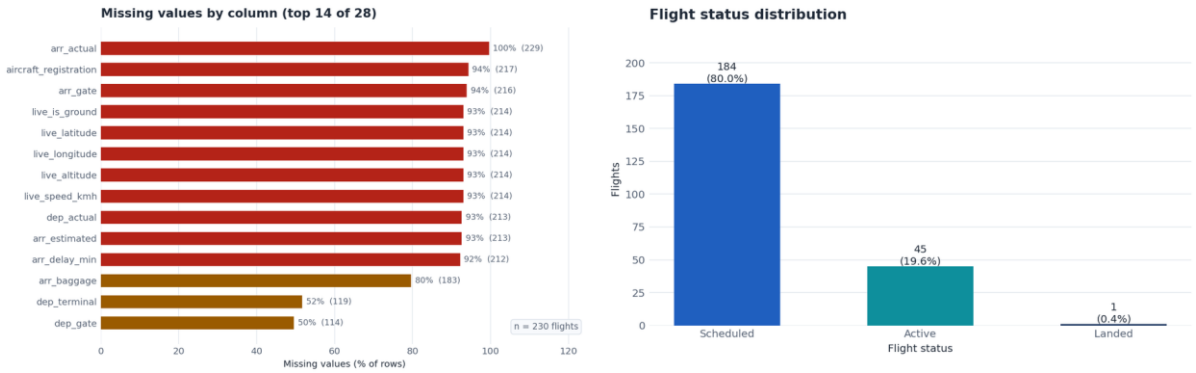


Figure 5. Left: missing values by column (top 13 of 28). Right: flight status distribution.

7.3 Descriptive Statistics

Table 12. Numeric summary computed from the database (`eda_summary.json`).

Column	n	Mean	Std	Min	Median	Max	Skew
dep_delay_min	125	4.0	10.9	0	0.0	68	4.2
arr_delay_min	18	0.7	2.3	0	0.0	9	3.4
max_delay_min	125	4.0	10.9	0	0.0	68	4.2
scheduled_duration_min	187	180.4	133.0	20	135.0	735	1.8
live_speed_kmh	16	637.3	291.0	0	783.1	830	-1.6
live_altitude	16	7341.4	4065.2	0	8450.6	11575	-1.0

Scheduled block time has a median of 135 min and a mean of 180 min (right-skewed, skew 1.8). Delay is extremely skewed (skew 4.2): the median is 0 min, the mean 4.0 min and the maximum 68 min.

7.4 Distributions

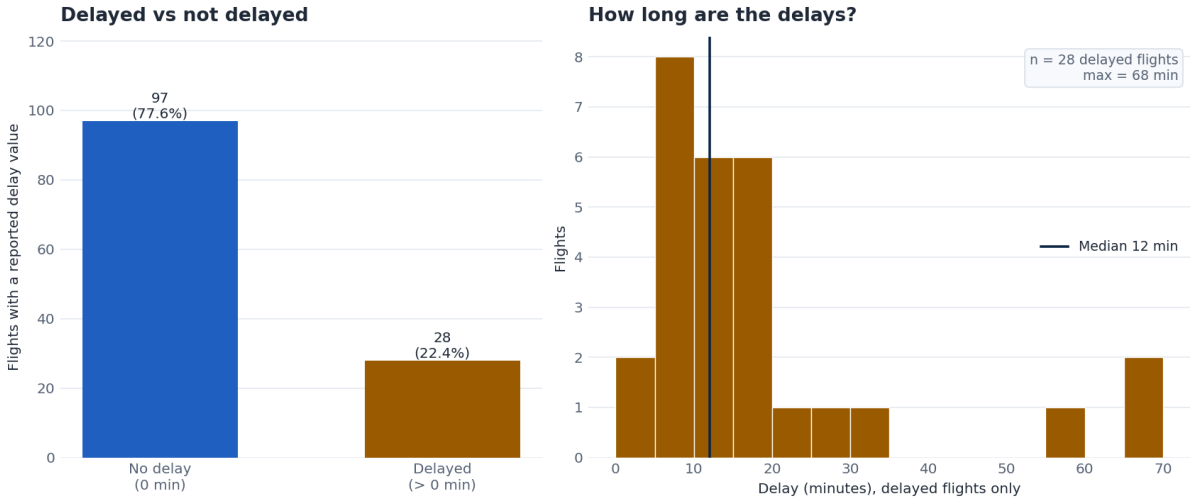


Figure 6. Delayed versus not delayed (left) and the size of the delays (right).

Of the 125 flights with delay data, **97 (77.6%) had no delay and 28 (22.4%) were delayed**. When a delay occurs it is typically **12 minutes** (median of delayed flights, quartiles 7.8 and 15.3); 4 delays reach 30 minutes or more and the longest is 68 minutes. 101 airlines appear and the largest, Japan Airlines, accounts for 5.2% of flights, so no carrier dominates; the most frequent departure airport

(TFU) has 28 flights (12.2%) and the 127 routes are spread thinly (most frequent BKK–SAI with 14 flights). Codeshare listings repeat the same physical flight under several airlines and therefore inflate airline and route counts. Scheduled durations form a main group below about 300 minutes and a smaller group between roughly 450 and 735 minutes (Figure 9, left).

7.5 Trends

Table 13. Delay behaviour by flight date (flights with delay data only).

Date	Flights	With delay data	Delayed	% delayed	Mean delay (min)
2026-10-09	132	66	9	13.6	2.45
2026-10-10	1	0	0	-	-
2026-10-13	97	59	19	32.2	5.64

The delayed share was 13.6% on 9 October and 32.2% on 13 October. With two usable dates this is a **difference between two snapshots, not a trend**; the dates also contain different airports and airlines, so no trend or seasonality claim is made. Departures concentrate in a morning block (08:00–12:00 local, peak 34 flights at 09:00) and an evening block (17:00–20:00, peak 36 flights at 19:00) (Figure 7). Among hours with at least five delay values the delayed share ranges from 0% (13:00 and 20:00) to 66.7% at 08:00, but these shares rest on 5–30 flights each and are confounded with airports and airlines (four of the 14 departures at 08:00 are Philippine Airlines flights, all delayed); they are indications for further data collection, not scheduling advice.

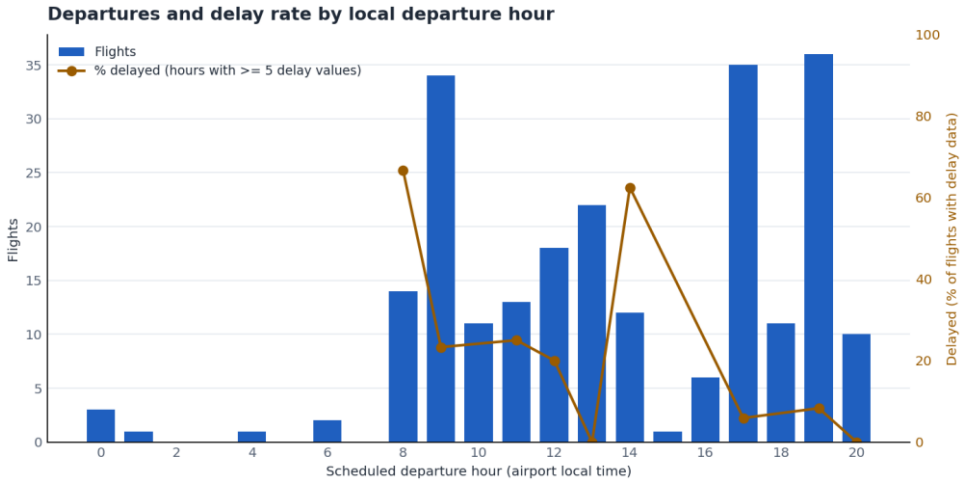


Figure 7. Departures and delay rate by local departure hour.

7.6 Relationships and Comparisons

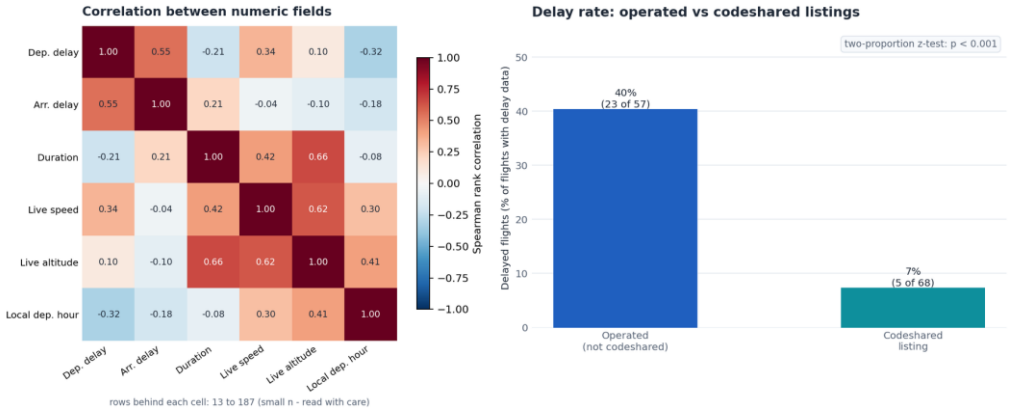


Figure 8. Left: Spearman correlation between numeric fields. Right: delay rate of operated flights versus codeshared listings.

In Figure 8, departure and arrival delay are positively related (rho = 0.55), but only 18 flights have both. Duration correlates with live altitude (0.66) and live speed (0.42), consistent with long flights cruising

high and fast, but these cells rest on very few tracked flights (13 to 187 rows per cell) and are illustrative only. Delay shows no strong relation with duration (−0.21). Operated flights were delayed in **23 of 57 (40%)** cases with delay data, codeshared listings in **5 of 68 (7%)** (two-proportion $z = 4.41$, $p < 0.001$). This is a real difference in this sample, but only an association: codeshare rows are marketing listings of a partner's flight, they duplicate physical flights, and delay values may be reported differently for them. It must not be read as "codesharing reduces delays".

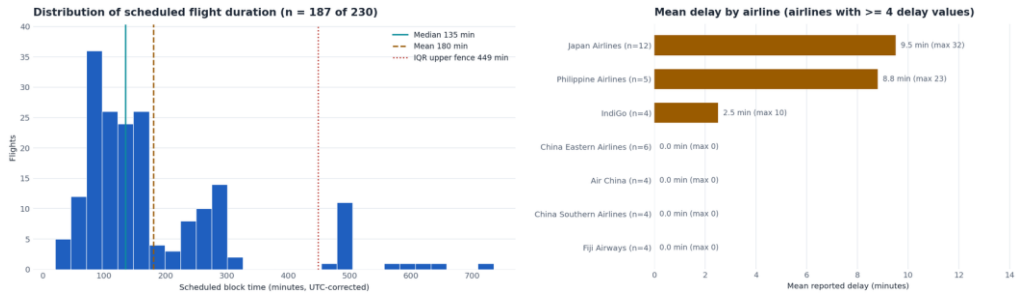


Figure 9. Left: distribution of scheduled flight duration (n = 187). Right: mean delay by airline (airlines with at least four delay values).

Japan Airlines (8 of 12 flights delayed, mean 9.5 min) and Philippine Airlines (4 of 5, mean 8.8 min) have the highest mean delays, while four other carriers show no delays in the flights that report them. At airport level, Lahore (LHW) has a mean of 9.3 min that stems from a single 65-minute delay among 7 flights, which shows that **means are dominated by single outliers and medians or delayed shares should be preferred**. Group sizes are 4 to 19 flights, so all rankings are descriptive.

7.7 Outliers



Figure 10. Outlier analysis (Tukey rule) for scheduled duration, delay of delayed flights and live ground speed.

Of 187 flights with a duration, **17 (9.1%) lie above the upper fence of 449 min** (maximum 735 min, Milan–Seoul); all are below the 24-hour validity limit, so they were kept and flagged instead of deleted. Before the timezone correction the same data showed a maximum of 1,155 min, so many apparent outliers were artefacts of the mislabelled timestamps. Among the 28 delayed flights (Figure 10) the fence is 26.5 min and 5 delays exceed it (27, 32, 55, 65 and 68 min); they are real events and are retained. Of the 16 live speed values, 2 are flagged; both are aircraft on the ground with speed 0 and are legitimate.

7.8 Key Findings

1. The database holds 230 validated flights on 3 flight dates; it is a snapshot of live data, not a time series.
2. Only 125 of 230 flights (54.3%) report a delay value; among them 28 (22.4%) were delayed, the median delay is 0 min, the mean 4.0 min and the maximum 68 min (skew 4.2). When a delay occurs it is typically 12 min.
3. Scheduled block time (timezone-corrected) has a median of 135 min; 17 flights (9.1%) lie above the IQR fence and were retained for review.

4. Operated flights were delayed more often than codeshared listings (40% of 57 versus 7% of 68, $p < 0.001$): an association in a small sample, not a cause.
5. Scheduled departures peak at 19:00 local time (36 flights); 101 airlines appear and none exceeds 5.2% of flights; live-tracking data exists for only 16 flights (7.0%).

Practical consequences. Delay should be reported as the share delayed plus the median among delayed flights, not as a mean over all flights, which hides both facts. Codeshare listings should be de-duplicated before airline or route statistics are computed (121 of 235 raw records are codeshare listings). Timestamps from this API must be converted from airport-local time before durations are computed. More capture dates and a plan that returns delay data for more flights are needed before the hour-of-day and airline differences can be tested; the pipeline supports this safely because re-runs only refresh existing rows.

8 Application Interfaces

8.1 Desktop Application

The PySide6 desktop application (`main_task/gui_main.py`) has 14 pages, grouped in a sidebar under Travel, Operations, and Data and System (Table 14). Long operations run on worker threads, and every table and chart is labelled by data origin (LIVE, HISTORICAL or SAMPLE DATA). The application starts and is fully usable without any key: Flight Search then runs in a clearly labelled SAMPLE mode with fictional carriers.

Table 14. Pages of the desktop application.

Page	Purpose
Home Dashboard	KPIs, status mix, data status and quick-action tiles
Flight Search	route, date, passengers and cabin search through FlightAPI or SAMPLE data; filters, sorting, detail view, saved searches, export
Flight Status and Delays	live status lookup (LIVE) and delay analytics over stored snapshots (HISTORICAL)
Airport Analytics; Airline Performance	traffic and detail views per airport; airline ranking with sample sizes and side-by-side comparison
Flight Route Map	offline map with real coordinates, hover tooltips and click filtering of the 127 stored routes
Destination Weather	forecast, current and historical weather for an airport and date (Open-Meteo)
Flight Analytics; Flight Explorer	status mix, airline activity, scheduling and live traffic; search, filter and inspect every stored flight
ETL Monitoring	run history, data-quality indicators, rejected records, redacted log, refresh of all sources
Reports and Export; Data Management	CSV / Excel / PDF of exactly the selected rows with filters and sources; retrieval with filters, database status, export
Settings and API; About	FlightAPI key (masked), animation switch, connection tests; version and credits



Figure 11. Home Dashboard, rendered from the delivered database (230 flights, 134 airports).

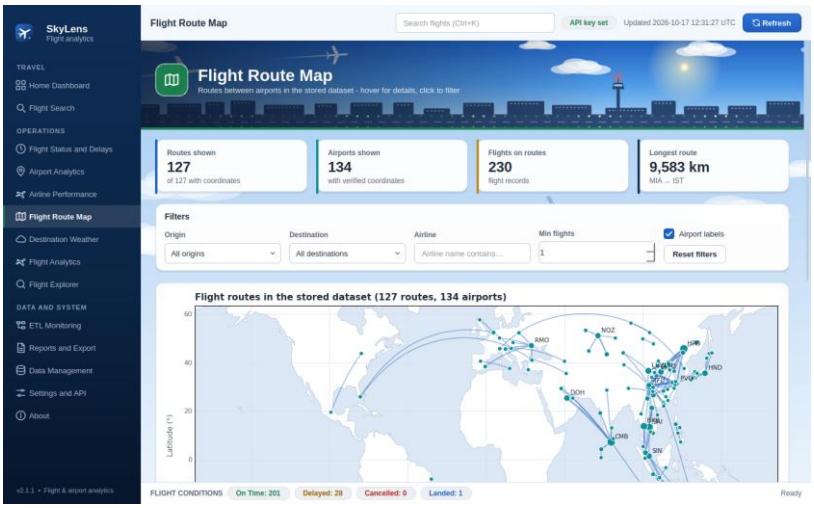


Figure 12. Flight Route Map showing the 127 stored routes with real airport coordinates.

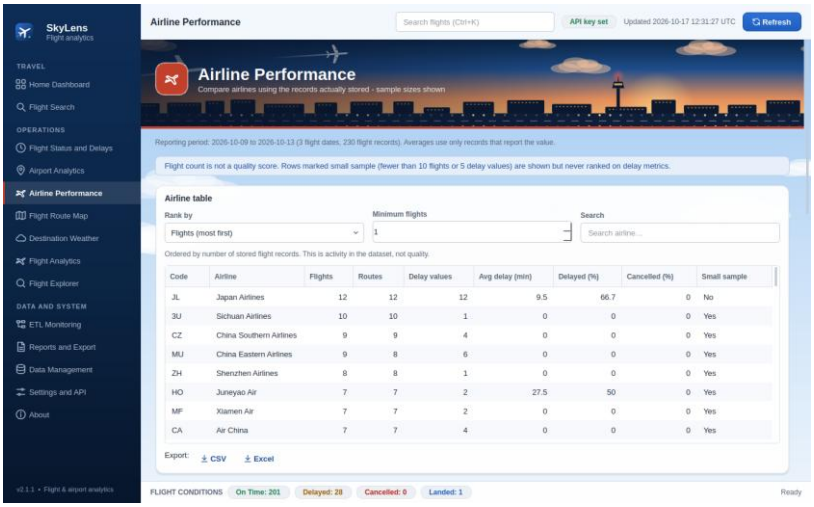


Figure 13. Airline Performance with sample sizes and small-sample flags.

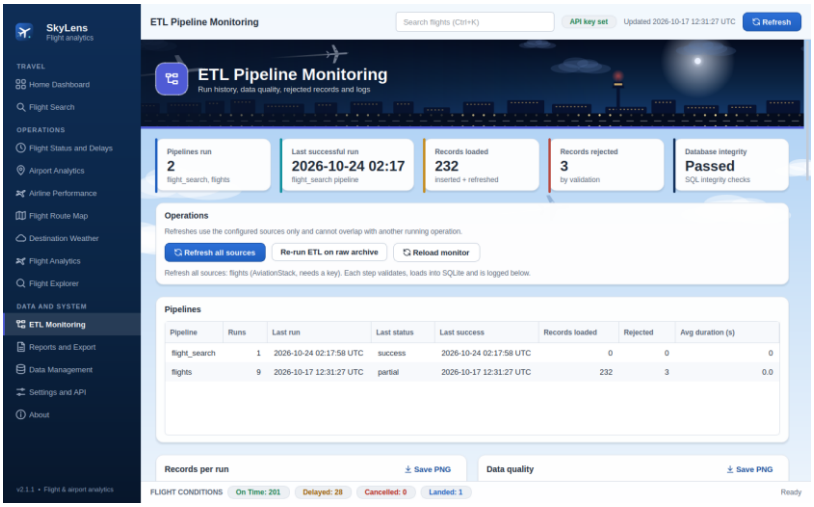


Figure 14. ETL Monitoring. The monitor also lists one SAMPLE-mode Flight Search run created while the screenshots were generated.

The Home Dashboard states “Delayed 28, 12.2% of stored flights” because it divides by all 230 stored flights, whereas the EDA reports 22.4% because it divides by the 125 flights that report a delay. Both are correct for their definitions; the EDA definition is the more meaningful one, and the Airline Performance page marks groups with fewer than 10 flights or 5 delay values as small samples that are never ranked on delay metrics.

8.2 Responsive Layout and Performance

The layout adapts to the window width (Figure 15): full sidebar from 1,020 px, icon sidebar from 700 to 1,019 px and a slide-in menu below 700 px, with forms and cards reflowing into one column; automated tests check that no page scrolls sideways at 380, 900 and 1,360 px. Version 2.1.1 removed freezes when switching menus. According to the profiling notes in `docs/PERFORMANCE.md`, pages are now rebuilt only when their data changed, charts are drawn one at a time from a render queue, pages are built lazily, and airport reference data and weather are cached; revisiting the four formerly unguarded pages dropped from 150–300 ms to 9–21 ms (headless run on a Linux sandbox, so absolute values differ per machine).

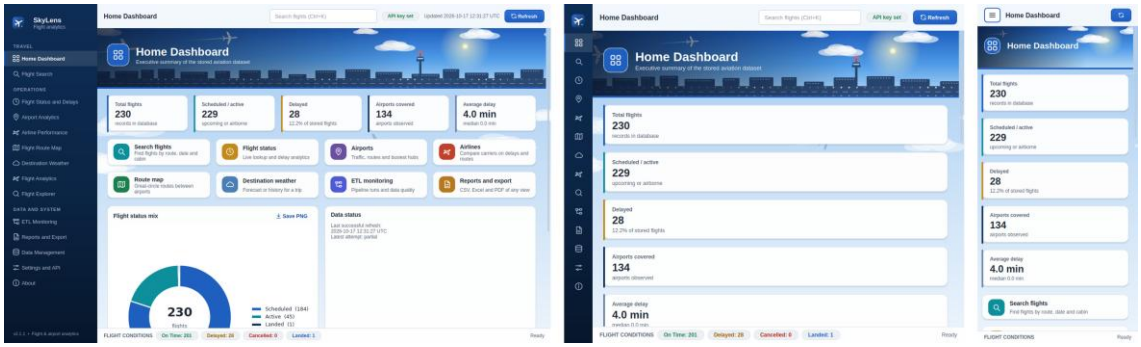


Figure 15. Home Dashboard at desktop (1,360 px), tablet (900 px) and phone (390 px) window widths.

Terminal menu and command line. `main_task/main.py` offers 12 actions (data refresh, record views, search, statistics, EDA, visualisations, database information, export, configuration, and ETL history with integrity checks), and `run_pipeline.py` and `run_eda.py` give scriptable access to the same functions. All exports (CSV, Excel, PDF) contain exactly the rows on screen together with filters, period, sources and limitations.

9 Testing and Verification

9.1 Automated Tests

The suite contains **276 tests, all passing** (Python 3.12 with the Qt offscreen platform, about 75 seconds). Tests use temporary databases and directories and never touch project data; no network access or API key is needed. Table 15 groups them by area.

Table 15. Automated tests by file.

File(s)	Tests	Covers
`test_travel_backend.py`	67	FlightAPI request and parsing (price blocks ignored), errors, retries, key scrubbing, sample provider, search pipeline, saved searches, schema migrations, analytics, weather, status, exports
`test_validation_and_transform.py`, `test_validation.py`	32	each Pydantic rule with its field-level reason, timezone-aware ordering, gate 2, timezone conversion, filling and unresolved handling
`test_gui_travel.py`, `test_gui_smoke.py`	43	Flight Search workflows, export equals displayed rows, operations pages, settings, headless GUI, threading
`test_performance.py`	41	lazy pages, reload guards, render queue, caches, vectorised airline table equals the old loop, HTTP timeouts, idle CPU
`test_pipeline.py`	19	end-to-end runs, idempotent re-run, persisted rejections, atomic rollback, failure logging, retries, key redaction, pagination
`test_gui_redesign.py`	17	layouts at three widths, no sideways scrolling on 14 pages, phone menu, hero banners, animation switch
`test_database.py`, `test_database_v2.py`	16	constraints, upsert semantics, migration, integrity verification
`test_analytics.py`, `test_data_processor.py`, `test_regressions.py`, `test_eda.py`	41	statistics and chart builders, flattening and derived columns, earlier bug fixes, EDA from SQLite, IQR outliers, z-test, generated findings

9.2 End-to-End Verification

- `run_pipeline.py --replay-raw --rebuild` reproduces **230 flights and 134 airports** from the four archived responses (three rejections in the last one) and the integrity checks pass.
- A second pass over the archive inserts **0** new rows and refreshes every accepted record (Table 10).
- `run_eda.py` regenerates the 12 figures, the summary JSON and the findings; the summary is identical to the delivered one.
- The GUI screenshots in Section 8 were rendered offscreen from the delivered code and database.

9.3 What Could Not Be Verified

No live call to AviationStack, FlightAPI or Open-Meteo could be made during the final verification, because the environment had no outbound access; the extraction stage was therefore verified with mocked HTTP responses (timeouts, 5xx, authentication errors, empty and HTML bodies, pagination) and indirectly by the real responses archived during development. The FlightAPI parser was verified against a payload shaped like its documentation only. The narrow “tablet” and “phone” layouts are window sizes of the desktop application; it was not run on physical devices, nor on Windows or macOS, and the Docker image was not built.

10 Limitations

- **Snapshot, not a time series:** three flight dates (one with a single flight) from a few API calls; no trend or seasonality claims are possible.
- **Partial delay data:** only 54.3% of flights report a delay and the API gives no causes of delay.
- **Codeshare duplication:** 121 of 235 raw records are codeshare listings, which inflates counts of airlines, routes and flights.

- **Small groups:** airline, airport and hour comparisons rest on 4 to 30 flights per group and are descriptive only.
- **Timestamp proof** covers departures from Asia (UTC+8 and UTC+9); 43 flights keep NULL times because no timezone was available, and nothing was guessed.
- **Live services unverified** in the final environment (Section 9.3); the delivered `.env` contains the AviationStack key, which should be revoked or replaced after submission.

11 Future Enhancements

The following items are ideas and **not** features of the current code: scheduled automatic retrievals to build a multi-day history within the quota; a mocked-API integration test of the GUI refresh path; enough data to test the hour-of-day and carrier differences statistically, including de-duplication of codeshare listings; a price-free comparison view of flights by duration and stops; and tests on physical tablets, phones and other operating systems.

12 Conclusion

SkyLens implements the complete ETL and EDA cycle on real flight data. Records are extracted with retries, explicit failure handling and a redacted, environment-based credential; every record passes two Pydantic gates with a persisted rejection trail; the pandas transformations are documented and justified; loading is an atomic, idempotent upsert verified by SQL; and the EDA reads from the database and reports distributions, relationships, comparisons and outliers with honest caveats. Cross-checking the API against its own live data exposed a systematic timestamp error that would have corrupted every cross-timezone duration; it was corrected rather than ignored. The desktop application makes the same data explorable through 14 responsive pages, and 276 automated tests pass. Every number in this report can be traced to an artefact in the project folder.

References

[1] apilayer, “Aviationstack API documentation.” [Online]. Available: https://aviationstack.com/documentation

[2] Pydantic, “Pydantic documentation.” [Online]. Available: https://docs.pydantic.dev

[3] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, 2010, pp. 56–61.

[4] SQLite, “About SQLite.” [Online]. Available: https://www.sqlite.org/about.html

[5] The Qt Company, “Qt for Python (PySide6) documentation.” [Online]. Available: https://doc.qt.io/qtforpython-6/

[6] FlightAPI, “One-way trip API documentation.” [Online]. Available: https://www.flightapi.io/documentation/oneway-trip-api/

[7] Open-Meteo, “Free weather API documentation.” [Online]. Available: https://open-meteo.com/en/docs

[8] OurAirports, “Open data downloads.” [Online]. Available: https://ourairports.com/data/

[9] Natural Earth, “Free vector and raster map data.” [Online]. Available: https://www.naturalearthdata.com

[10] Python Software Foundation, “Python 3 documentation.” [Online]. Available: https://docs.python.org/3/

[11] K. Reitz, “Requests: HTTP for Humans.” [Online]. Available: https://requests.readthedocs.io

[12] S. Kumar, “python-dotenv.” [Online]. Available: https://github.com/theskumar/python-dotenv

[13] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[14] pytest-dev, “pytest documentation.” [Online]. Available: https://docs.pytest.org

[15] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases.” [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[16] Python Software Foundation, “zoneinfo — IANA time zone support.” [Online]. Available: https://docs.python.org/3/library/zoneinfo.html